دالة التحقق من العدد الأولي

```cpp
int is_prime(long long n)
{
    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    if (n % 2 == 0)
        return 0;

    for (long long i = 3; i <= n / i; i += 2)
    {
        if (n % i == 0)
            return 0;
    }

    return 1;
}
```

شرح الفكرة

1. إذا كان `n < 2` فالعدد ليس أوليًا.
2. العدد `2` أولي، وهو العدد الزوجي الأولي الوحيد.
3. إذا كان العدد زوجيًا وغير 2، فهو ليس أوليًا.
4. نختبر القسمة على الأعداد الفردية بدايةً من 3.
5. إذا وجدنا قاسمًا دون باقٍ، نرجع `0` ، أي ليس أوليًا.
6. إذا انتهت الحلقة دون العثور على قاسم، نرجع `1` ، أي أولي.

لماذا لا نختبر حتى العدد نفسه؟

يكفي اختبار القواسم حتى الجذر التربيعي للعدد؛ لأن أي عدد مركب لا بد أن يكون له قاسم لا يتجاوز هذا الجذر.

استخدمنا الشرط:

```
i ⩽ n / i
```

بدلًا من:

```
i * i ⩽ n
```

لتجنب احتمال حدوث overflow عند ضرب أعداد كبيرة.

معنى باقي القسمة `%`

```
n % i == 0
```

يعني أن `n` يقبل القسمة على `i` دون باقٍ.

معنى `return`

- `return 1;` تعني أن العدد أولي.
- `return 0;` تعني أن العدد ليس أوليًا.

Tests

```
int main(void)
{
    int count;

    printf("How many numbers do you want to test? ");

    if (scanf("%d", &count) ≠ 1 || count ⩽ 0)
```

```c
    {
        printf("Invalid input.\n");
        return 1;
    }

    long long *numbers =
        malloc((size_t)count * sizeof(long long));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < count; i++)
    {
        printf("Enter number %d: ", i + 1);

        if (scanf("%lld", &numbers[i]) != 1)
        {
            printf("Invalid input.\n");
            free(numbers);
            return 1;
        }
    }

    printf("\nPrime Number Test Results:\n");

    for (int i = 0; i < count; i++)
    {
        if (is_prime(numbers[i]))
            printf("%lld → Prime\n", numbers[i]);
        else
            printf("%lld → Not prime\n", numbers[i]);
    }

    free(numbers);

    return 0;
```

```
}
```

- `count` : عدد الأرقام المطلوب اختبارها.
- `malloc` : تحجز مساحة في الذاكرة حسب عدد الأرقام.
- `numbers` : مؤشر إلى بداية المساحة المحجوزة.
- `sizeof(long long)` : حجم العنصر الواحد بالبايت.
- `free(numbers)` : تحرير الذاكرة بعد الانتهاء.

يجب التأكد أن الإدخال صحيح وأن `count > 0` ، والتحقق من نجاح `malloc` قبل استخدام المصفوفة.

قراءة الأرقام وطباعة النتائج

```
for (int i = 0; i < count; i++)
{
    scanf("%lld", &numbers[i]);
}

for (int i = 0; i < count; i++)
{
    if (is_prime(numbers[i]))
        printf("%lld → Prime\n", numbers[i]);
    else
        printf("%lld → Not prime\n", numbers[i]);
}
```

- `%d` لقراءة عدد من النوع `int` .
- `%lld` لقراءة أو طباعة عدد من النوع `long long` .
- `numbers[i]` للوصول إلى العنصر رقم `i` في المصفوفة.
- `&numbers[i]` لإرسال عنوان العنصر إلى `scanf` حتى تخزن القيمة داخله.

RUN

```
gcc main.c -o prime.exe
```

```
.\prime.exe
```

```
D:\IT\هيلك\Prime_Number>.\prime.exe
How many numbers do you want to test? 3
Enter number 1: 20
Enter number 2: 7
Enter number 3: 2

Prime Number Test Results:
20 -> Not prime
7 -> Prime
2 -> Prime
```

التعقيد الزمني (Time Complexity)

Time Complexity: $O(\sqrt{n})$

Space Complexity للدالة: $O(1)$

Space Complexity للبرنامج: $O(k)$

حيث k هو عدد الأرقام المخزنة في المصفوفة.